

# **Data Mastery 120: Roadmap Integral de Minería y Análisis de Datos**

***Día 1 – Conexión y Carga (Dataset: IBM HR)***



## Objetivo del Día 1

- Conectar el dataset IBM HR a tu entorno SQL (PostgreSQL + DBeaver).
- Explorar las primeras filas para validar la carga.
- Construir un esquema ER inicial del dataset.



## Normalización de Columnas (snake\_case)

Antes de cargar el dataset en SQL, normalizamos los nombres de las columnas para evitar conflictos de mayúsculas/minúsculas.

"""

TÍTULO: Pipeline de Normalización y Limpieza de Datos de RRHH

OBJETIVO: Automatizar la lectura de reportes de Recursos Humanos, estandarizar los

nombres de las columnas a formato 'snake\_case' y exportar una versión limpia con

marca de tiempo, asegurando la compatibilidad con Excel y el orden histórico.

"""

# Herramientas para el manejo de datos y archivos CSV

import pandas as pd

import csv

# Utilidades para interactuar con las carpetas y rutas del sistema operativo

import os

from pathlib import Path

# Funciones para limpieza de texto (expresiones regulares) y gestión de fechas

import re

from datetime import datetime

# -----

# 1. Configuración de rutas.

# -----

# --- Configuración de rutas del proyecto ---

# Localiza la carpeta raíz subiendo tres niveles y define las rutas de entrada (raw)

# y salida (processed) para mantener los datos organizados.

base\_dir = Path(\_\_file\_\_).resolve().parents[2]

target\_raw\_dir = base\_dir / '02\_data' / 'raw' / 'rrhh'

cleaned\_target\_dir = base\_dir / '02\_data' / 'processed' / 'rrhh'

# --- Identificación del archivo de origen ---

# Define el nombre específico del archivo de RRHH y construye su ruta

# completa para que el sistema pueda localizarlo.

```

rrhh_file_name = 'WA_Fn-UseC_-HR-Employee-Attrition'
rrhh_path_csv = os.path.join(target_raw_dir, f'{rrhh_file_name}.csv')
# --- Generación del archivo de salida con marca de tiempo ---
# Crea un nombre único para el archivo procesado usando la fecha y hora
actual.
# Esto evita que los resultados nuevos sobrescriban a los anteriores.
time_stamp = datetime.now().strftime('%Y%m%d_%H%M%S')
cleaned_file_name = f'rrhh_limpio_{time_stamp}.csv'
path_csv = os.path.join(cleaned_target_dir, cleaned_file_name)

# -----
# 2. Normalización de Nombres de Columnas.
# -----
def to_snake_case(name):
    """
    Convierte un texto de formato CamelCase o PascalCase a snake_case.
    """
    # Inserta un guion bajo entre una letra minúscula y una mayúscula que
    # inicia una palabra.
    # Ejemplo: "EmpleadoId" -> "Empleado_Id"
    s1 = re.sub('([A-Z][a-z]+)', r'\1_\2', name)
    # Maneja casos con números o mayúsculas consecutivas y convierte todo
    # a minúsculas.
    # Ejemplo: "Empleado_Id" -> "empleado_id"
    return re.sub('([a-z0-9])([A-Z])', r'\1_\2', s1).lower()

# -----
# 3. Carga del Dataset.
# -----
try:
    # --- Carga y normalización de datos ---
    # Importamos el archivo CSV a un formato de tabla (DataFrame) y
    # transformamos
    # todos los nombres de las columnas a 'snake_case' para estandarizar el
    # formato.
    df = pd.read_csv(rrhh_path_csv)
    df.columns = [to_snake_case(col) for col in df.columns]
    # --- Preparación para la exportación ---
    # Convertimos la tabla en una lista de diccionarios y extraemos los
    # nombres
    # de las columnas; esto facilita la escritura posterior del archivo limpio.
    datos_dict = df.to_dict(orient='records')
    mis_cabeceras = df.columns.tolist()
except Exception as e:

```

# Gestión de errores en caso de que el archivo no exista o el formato sea incorrecto.

```
print(f"❌ Error al cargar o procesar el dataset: {e}")
```

```
datos_dict = None
```

# -----

# 4. Exportación de Datos.

# -----

```
def exportar_a_csv(ruta_destino, datos, cabeceras):
```

```
    """
```

Gestiona la creación de carpetas y guarda los datos procesados en un archivo CSV.

```
    """
```

```
    if datos is None:
```

```
        return
```

```
    try:
```

```
        # --- Preparación del entorno ---
```

```
        # Verifica si existe la carpeta de destino; si no, la crea automáticamente
```

```
        # para evitar errores al intentar guardar el archivo.
```

```
        if not os.path.exists(cleaned_target_dir):
```

```
            os.makedirs(cleaned_target_dir, exist_ok=True)
```

```
            print(f"✅ Carpeta creada en: {cleaned_target_dir}")
```

```
        # --- Escritura del archivo ---
```

```
        # Abre el archivo con codificación 'utf-8-sig' para que Excel reconozca
```

```
        # correctamente tildes y eñes. Se usa el delimitador '|' para mayor
```

claridad.

```
        with open(ruta_destino, mode='w', newline="", encoding='utf-8-sig') as  
        archivo_csv:
```

```
            df.to_csv(archivo_csv, sep='|', index=False)
```

```
        # --- Confirmación y resumen ---
```

```
        # Muestra en consola un resumen técnico de los datos guardados y
```

```
        # confirma la ubicación final del archivo.
```

```
        print(df.info())
```

```
        print(f"\n✅ Proceso completado exitosamente.")
```

```
        print(f"📁 Archivo guardado en: {ruta_destino}")
```

```
        print(f"📄 CSV: {cleaned_file_name}")
```

```
    except PermissionError:
```

# Error común si el archivo de destino está abierto en Excel o por otro usuario.

```
        print(f"❌ Error de Permiso: El archivo '{ruta_destino}' está abierto en  
        otro programa.")
```

```
    except Exception as e:
```

```

    # Captura cualquier otro imprevisto durante la exportación.
    print(f" ✖ Ocurrió un error inesperado al exportar: {e}")
# --- Punto de entrada del script ---
# Asegura que el proceso de limpieza y exportación solo se inicie si el
# archivo se ejecuta directamente (no si se importa desde otro script).
if __name__ == "__main__":
    print("--- Iniciando proceso de normalización ---\n")
    exportar_a_csv(path_csv, datos_dict, mis_cabeceras)

```

## Ejemplo de columnas normalizadas:

- EmployeeNumber → employee\_number
- Attrition → attrition
- JobRole → job\_role
- MonthlyIncome → monthly\_income
- PerformanceRating → performance\_rating
- YearsAtCompany → years\_at\_company
- EducationField → education\_field

## Guía Detallada Paso a Paso

### Paso 1. Instalación de PostgreSQL & DBeaver

- Descarga **PostgreSQL** desde el sitio oficial de **PostgreSQL** (<https://www.postgresql.org/download/>).
- Instala **DBeaver** para administrar la base (<https://dbeaver.io/download/>).

### Paso 2. Preparación del Entorno

Abrir **DBeaver** → Cliente gráfico para gestionar **PostgreSQL**.

Crear conexión a **PostgreSQL**:

- Menú: **Database** → **New Database Connection**.
- Selecciona **PostgreSQL**.
- Configura:
  - **Host**: localhost
  - **Port**: 5432
  - **Database**: postgres (por defecto)
  - **User**: postgres
  - **Password**: tu contraseña definida.
- Haz clic en **Test Connection** → debe mostrar **Success**.
- **Guarda** la conexión.



### Paso 3. Crear la Base de Datos ibm\_hr

En el editor **SQL** de **DBeaver**:

- Abre una nueva consulta y ejecuta:

```

/*
-----
TÍTULO: Creación de la base de datos para Recursos Humanos de IBM
OBJETIVO: Crear un contenedor lógico (base de datos) para almacenar
toda la
        información relacionada con el personal, departamentos y
nómina.
DESCRIPCIÓN: Este script inicializa el entorno de trabajo ejecutando la
sentencia
        DDL (Data Definition Language) necesaria para generar el
esquema
        principal de la base de datos 'ibm_hr'.
-----
*/

-- 1. Sentencia principal para crear la base de datos
-- Esta línea genera una nueva base de datos vacía llamada 'ibm_hr'.
CREATEDATABASE ibm_hr;

-- NOTA OPERATIVA: Una vez creada, asegúrate de activar la base de
datos
-- (Set as default) para que todas las tablas posteriores se guarden en
este proyecto.

```

- **Check de Ingeniería:**
  - Confirmar que el nombre está en **snake\_case**.
  - Validar que la base aparece en el panel izquierdo.



### Paso 4. Crear la Tabla Principal employee\_master\_data

Selecciona la base **ibm\_hr**.

Ejecuta el script de creación de tabla con columnas normalizadas:

```

/*****
*****
TITULO: Creación de Tabla Maestra de Empleados
OBJETIVO: Establecer la estructura base para almacenar la información
demográfica,

```

organizacional y de compensación de los colaboradores.  
DESCRIPCIÓN: Este script crea la tabla 'employee\_master\_data' si no existe,

incluyendo campos clave para análisis de retención, movilidad, estructura de puestos y compensaciones (sueldos y opciones).

\*\*\*\*\*  
\*\*\*\*\*/

**CREATE TABLE IF NOT EXISTS employee\_master\_data(**

--

=====

=====

-- 1. DATOS DEMOGRÁFICOS Y PERSONALES

-- Información básica del colaborador.

--

=====

=====

age INT NOT NULL, -- Edad del colaborador (obligatorio).

gender VARCHAR(50), -- Género registrado.

marital\_status VARCHAR(20), -- Estado civil (Single, Married, Divorced).

over18 CHAR(1), -- Verificación de mayoría de edad (Y/N).

--

=====

=====

-- 2. INDICADORES DE RETENCIÓN Y MOVILIDAD

-- Métricas clave para análisis de fuga de talento (Attrition) y traslados.

--

=====

=====

attrition VARCHAR(10), -- Abandono de la empresa (Yes/No).

business\_travel VARCHAR(50), -- Frecuencia de viajes laborales.

distance\_from\_home INT, -- Distancia al centro de trabajo en km.

--

=====

=====

-- 3. ESTRUCTURA ORGANIZACIONAL Y PUESTO

-- Ubicación funcional del empleado.

--

=====

=====

department VARCHAR(50), -- Área o departamento funcional.

```

job_role VARCHAR(50),          -- Nombre del cargo o función.
job_level INT,                 -- Nivel de jerarquía en la organización.
employee_number INT PRIMARY KEY, -- Identificador único (Llave
Primaria).
employee_count INT,           -- Conteo unitario (usualmente 1 por
fila).

--
=====
=====
-- 4. COMPENSACIÓN Y BENEFICIOS
-- Datos financieros relacionados con el empleo.
--
=====
=====
monthly_income INT,            -- Sueldo mensual bruto.
monthly_rate INT,              -- Tasa o costo mensual relativo.
daily_rate INT,                -- Pago por jornada diaria.
hourly_rate INT,               -- Pago por hora laborada.
percent_salary_hike INT,       -- Porcentaje del último incremento.
stock_option_level INT,        -- Nivel de opciones sobre acciones.

--
=====
=====
-- 5. Educación y Formación
-- Historial académico y preparación técnica.
--
=====
=====
education INT,                 -- Nivel académico alcanzado (codificado).
education_field VARCHAR(50),   -- Especialidad o carrera
estudiada.
training_times_last_year INT,  -- Cursos de capacitación tomados
el año pasado.

--
=====
=====
-- 6. Encuestas de Satisfacción y Desempeño
-- Percepción del empleado y calidad de su trabajo (Escala
numéricas).
--
=====
=====

```



```

environment_satisfaction INT,      -- Satisfacción con el entorno
laboral.
job_involvement INT,              -- Nivel de compromiso con el puesto.
job_satisfaction INT,             -- Nivel de felicidad en el trabajo.
performance_rating INT,          -- Calificación de la última evaluación.
relationship_satisfaction INT,    -- Calidad de relación con colegas.
work_life_balance INT,           -- Equilibrio entre vida y trabajo.

--
=====
=====
-- 7. Trayectoria y Permanencia
-- Experiencia acumulada dentro y fuera de la organización.
--
=====
=====
num_companies_worked INT,         -- Cantidad de empresas previas.
total_working_years INT,         -- Años totales de experiencia laboral.
years_at_company INT,            -- Antigüedad en la empresa actual.
years_in_current_role INT,       -- Tiempo en el puesto actual.
years_since_last_promotion INT,  -- Años desde el último ascenso.
years_with_curr_manager INT,     -- Tiempo bajo el mando del jefe
actual.

--
=====
=====
-- 8. Condiciones Laborales
-- Parámetros finales sobre la jornada de trabajo.
--
=====
=====
over_time VARCHAR(10),           -- Realización de horas extra
(Yes/No).
standard_hours INT-- Horas base establecidas por contrato.
);

```

- **Pitfall común:** olvidar ";" al final o usar **nombres con espacios**.
- **Check de Ingeniería:** todos los nombres en **snake\_case**, tipos de datos correctos.



## Paso 5. Importar el Dataset IBM HR (Bulk Load)

Antes de **normalizar**, necesitamos los **datos** dentro. Usaremos el **asistente de DBeaver** para mayor precisión:

Opción 1:

1. Menú: **Tools** → **Data Transfer** → **Import Data**.
2. Selecciona **archivo CSV** del dataset **IBM HR**.
3. Destino: **tabla employee\_master\_data**.
4. Configura **delimitador** , y **encabezados**.
5. **Ejecuta** la **importación**.

Opción 2:

1. En **DBeaver**, haz clic derecho sobre la **tabla employee\_master\_data** creada ayer.
2. Selecciona **"Import Data"** -> **CSV**.
3. Busca tu **archivo de IBM HR**.
4. **Mapeo**: Asegúrate de que las **columnas** del CSV coincidan con los nombres de la tabla.  
**Ejecutar**: Una vez finalizado, **verifica** que los **datos** se **cargaron** correctamente.



## Paso 6. Controles de la calidad de los datos (employee\_master\_data)

- **Confirma que los datos se cargaron correctamente:**

```
/*****  
*****/
```

Título: Visualización Preliminar de Datos Maestros de Empleados  
Objetivo: Obtener una muestra rápida de los datos almacenados en la  
tabla

principal de empleados.

Descripción: Selecciona las primeras 10 filas de la tabla  
'employee\_master\_data'

para verificar la estructura, el tipo de datos y la calidad de la  
información sin cargar toda la base de datos.

\*\*\*\*\*

\*\*\*\*\*/

-- Selecciona todas las columnas (\*) de la tabla especificada

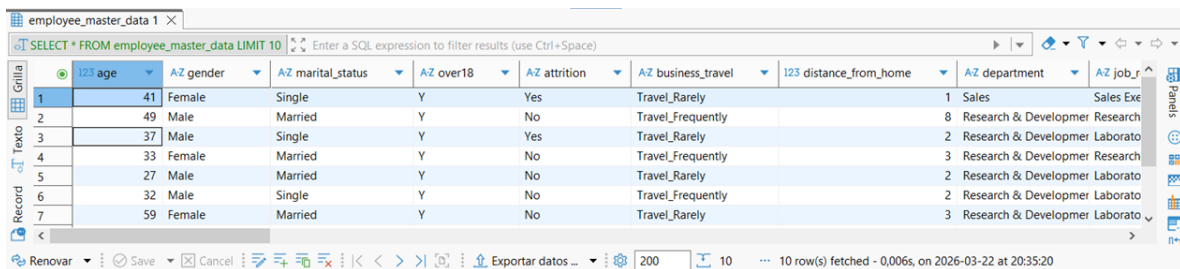
**SELECT \***

-- Define la tabla de origen de los datos

**FROM employee\_master\_data**

-- Limita el resultado a solo las primeras 10 filas para una vista rápida

**LIMIT 10;**



The screenshot shows a database interface with a table named 'employee\_master\_data'. The table has 10 columns: age, gender, marital\_status, over18, attrition, business\_travel, distance\_from\_home, department, and job\_role. The first 10 rows are displayed, showing a mix of employees with various attributes. The interface includes a search bar, a filter icon, and a status bar at the bottom indicating '10 row(s) fetched'.

|   | age | gender | marital_status | over18 | attrition | business_travel   | distance_from_home | department             | job_role              |
|---|-----|--------|----------------|--------|-----------|-------------------|--------------------|------------------------|-----------------------|
| 1 | 41  | Female | Single         | Y      | Yes       | Travel_Rarely     |                    | Sales                  | Sales Executive       |
| 2 | 49  | Male   | Married        | Y      | No        | Travel_Frequently |                    | Research & Development | Research Scientist    |
| 3 | 37  | Male   | Single         | Y      | Yes       | Travel_Rarely     |                    | Research & Development | Laboratory Technician |
| 4 | 33  | Female | Married        | Y      | No        | Travel_Frequently |                    | Research & Development | Research Scientist    |
| 5 | 27  | Male   | Married        | Y      | No        | Travel_Rarely     |                    | Research & Development | Laboratory Technician |
| 6 | 32  | Male   | Single         | Y      | No        | Travel_Frequently |                    | Research & Development | Laboratory Technician |
| 7 | 59  | Female | Married        | Y      | No        | Travel_Rarely     |                    | Research & Development | Laboratory Technician |

### ○ Detección de registros duplicados:

\*\*\*\*\*

\*\*\*\*\*/

TÍTULO: Detección de Duplicados en Maestro de Empleados

OBJETIVO: Identificar números de empleado que aparecen más de una vez en la tabla

maestra para asegurar la integridad de los datos.

DESCRIPCIÓN: Este script consulta la tabla principal de empleados, agrupa por el

identificador único y filtra aquellos grupos que tienen un conteo superior a 1, indicando una duplicidad de registros.

\*\*\*\*\*

\*\*\*\*\*/

**SELECT**

-- Selecciona el identificador del empleado

**employee\_number,**

-- Cuenta cuántas veces aparece cada número para identificar el total de duplicados

**COUNT(\*) AS repeticiones**

-- Define la tabla de origen de datos maestros de empleados

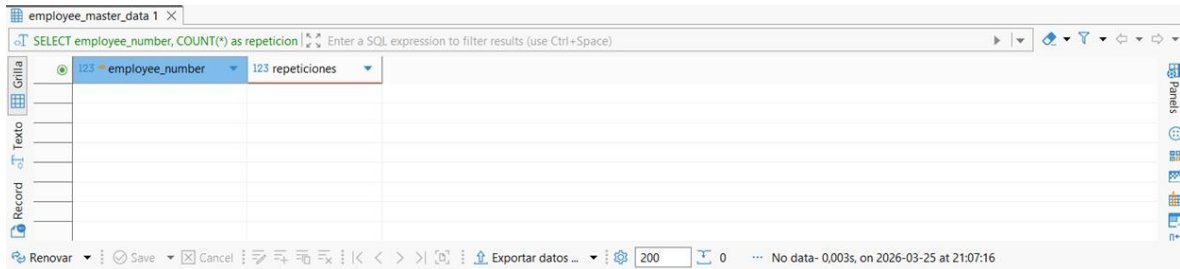
**FROM employee\_master\_data**

-- Agrupa los resultados por número de empleado para consolidar el conteo

**GROUP BY employee\_number**

-- Filtro final: Solo muestra los números de empleado cuyo conteo sea mayor a 1

**HAVING COUNT(\*) > 1;**



The screenshot shows a data grid interface with a table titled 'employee\_master\_data 1'. The table has two columns: 'employee\_number' and 'repeticiones'. The 'employee\_number' column has a dropdown menu showing '123' and 'repeticiones'. The 'repeticiones' column has a dropdown menu showing '123'. The table is empty. The interface includes a search bar at the top, a sidebar on the left with 'Grilla', 'Record', and 'Texto' buttons, and a bottom bar with 'Renovar', 'Save', 'Cancel', and 'Exportar datos ...' buttons. The bottom bar also shows '200' and '0'.

### ○ Identificación de Valores Nulos en Columnas Críticas:

```
/*
*****
TITULO: Depuración de Maestros de Empleados
OBJETIVO: Obtener una lista limpia de colaboradores desde la tabla
maestra.
DESCRIPCIÓN: Selecciona todos los registros de
'employee_master_data' que tengan
información crítica mínima, excluyendo registros donde el ID,
departamento, puesto o salario estén vacíos (NULL).
*****
*/
```

**SELECT \***

-- Tabla fuente con la información cruda de los colaboradores

**FROM employee\_master\_data**

-- CRITERIOS DE VALIDACIÓN: El registro debe tener contenido en AL MENOS

-- uno de estos campos para ser considerado válido y útil.

**WHERE**

**employee\_number IS NOT NULL OR**-- 1. Que el ID de empleado esté definido.

**department IS NOT NULL OR**-- 2. Que el departamento tenga un nombre asignado.

**job\_role IS NOT NULL OR**-- 3. Que el puesto de trabajo esté definido.

**monthly\_income IS NOT NULL;**-- 4. Que exista un registro del salario mensual.

| employee_master_data 1                      |         |           |                   |           |              |                    |                        |                          |         |
|---------------------------------------------|---------|-----------|-------------------|-----------|--------------|--------------------|------------------------|--------------------------|---------|
| SELECT * FROM employee_master_data WHERE em |         |           |                   |           |              |                    |                        |                          |         |
|                                             | 123 age | AZ gender | AZ marital_status | AZ over18 | AZ attrition | AZ business_travel | 123 distance_from_home | AZ department            | AZ job  |
| 1                                           | 41      | Female    | Single            | Y         | Yes          | Travel_Rarely      |                        | 1 Sales                  | Sales E |
| 2                                           | 49      | Male      | Married           | Y         | No           | Travel_Frequently  |                        | 8 Research & Development | Resear  |
| 3                                           | 37      | Male      | Single            | Y         | Yes          | Travel_Rarely      |                        | 2 Research & Development | Laborz  |
| 4                                           | 33      | Female    | Married           | Y         | No           | Travel_Frequently  |                        | 3 Research & Development | Resear  |
| 5                                           | 27      | Male      | Married           | Y         | No           | Travel_Rarely      |                        | 2 Research & Development | Laborz  |
| 6                                           | 32      | Male      | Single            | Y         | No           | Travel_Frequently  |                        | 2 Research & Development | Laborz  |
| 7                                           | 59      | Female    | Married           | Y         | No           | Travel_Rarely      |                        | 3 Research & Development | Laborz  |

## ○ Auditoría de Integridad (Valores fuera de rango o vacíos):

```

/*****
*****
TITULO: Detección de Colaboradores con Datos Incompletos o
Inválidos
OBJETIVO: Identificar empleados en la tabla maestra que carecen de
área asignada
o que tienen un sueldo ilógico (cero o negativo).
DESCRIPCIÓN: Este script consulta la base de datos de RRHH para
encontrar
registros con errores críticos en 'departamento' o 'ingreso
mensual'
que requieren corrección.
*****/

```

### SELECT

**employee\_number**, -- Identificador único del colaborador  
**department**, -- Área asignada (se busca si está vacía)  
**monthly\_income** -- Sueldo registrado (se busca si es incorrecto)  
-- Tabla principal que contiene la información de RRHH

### FROM employee\_master\_data

-- CRITERIOS DE ERROR: Filtra registros con problemas de calidad de datos

### WHERE

**department= ''** OR -- Error 1: Departamento vacío (texto sin contenido)  
**monthly\_income <= 0**; -- Error 2: Salario cero o negativo (valor ilógico)



- **Distribución de la plantilla por departamento:**

```

/*****
*****
TITULO: Reporte de Conteo de Empleados por Departamento
OBJETIVO: Obtener el total de empleados activos agrupados por su
área funcional.
DESCRIPCIÓN: Este script consulta la tabla maestra de empleados para
sumar cuántos
            empleados pertenecen a cada departamento, permitiendo
visualizar la
            distribución del personal.
*****
*****/

```

### SELECT

-- Selecciona la columna del departamento para identificar el grupo  
**department,**

-- Cuenta el número total de filas (empleados) dentro de cada grupo  
**COUNT(\*) AS total\_empleados**

-- Define la tabla principal donde se almacena la información de los empleados

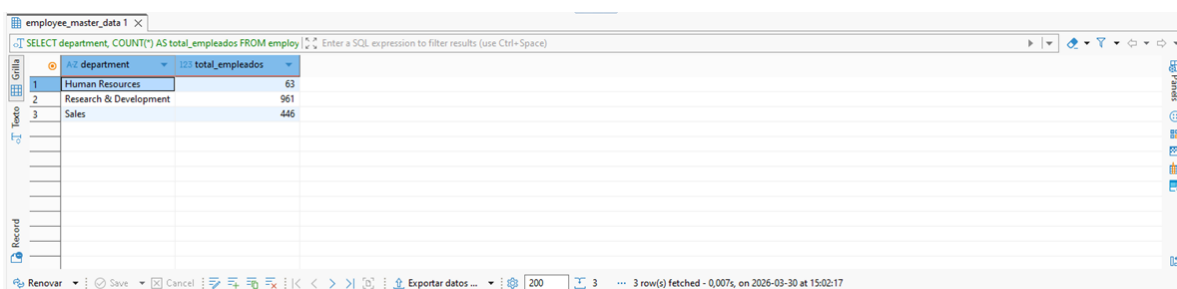
**FROM employee\_master\_data**

**GROUP BY**

-- Agrupa los resultados por departamento para que el conteo no sea global,

-- sino específico para cada área definida en la tabla.

**department;**



The screenshot shows a database interface with a query window and a results table. The query is: `SELECT department, COUNT(*) AS total_empleados FROM employee_master_data`. The results table has two columns: 'department' and 'total\_empleados'. It contains three rows of data.

|   | department             | total_empleados |
|---|------------------------|-----------------|
| 1 | Human Resources        | 63              |
| 2 | Research & Development | 961             |
| 3 | Sales                  | 446             |

- **Pitfall común:** olvidar **GROUP BY** cuando usas agregaciones.
- **Check de Ingeniería:** validar que los resultados coinciden con expectativas (ej. número total de empleados).



## Paso 7. Construir Esquema ER (Entity-Relationship) en DBeaver

En **DBeaver**:

Opción 1:

- Vaya al **Explorador de proyectos** (normalmente a la izquierda).
- Haga **clic derecho** en la carpeta **Diagramas** (Diagrams).
- Seleccione **Crear nuevo diagrama**.
- Seleccionar **Base de Datos**: Póngale un **nombre** al diagrama (ej. **ibm\_hr\_erd**) y busque la base de datos **ibm\_hr**.
- Selecciona la tabla **employee\_master\_data**.
- **Clic** en **Finalizar**.
- También puede **arrastrar y soltar tablas** directamente desde el navegador de bases de datos hacia el **lienzo** del diagrama.
- **Generación Automática**: DBeaver visualizará automáticamente las **relaciones** existentes basadas en las **claves foráneas (FK)** y **primarias (PK)**.
- **Exporta** como **imagen** o **PDF** para tu portafolio.



| employee_master_data           |
|--------------------------------|
| 123 employee_number            |
| 123 age                        |
| AZ gender                      |
| AZ marital_status              |
| AZ over18                      |
| AZ attrition                   |
| AZ business_travel             |
| 123 distance_from_home         |
| AZ department                  |
| AZ job_role                    |
| 123 job_level                  |
| 123 employee_count             |
| 123 monthly_income             |
| 123 monthly_rate               |
| 123 daily_rate                 |
| 123 hourly_rate                |
| 123 percent_salary_hike        |
| 123 stock_option_level         |
| 123 education                  |
| AZ education_field             |
| 123 training_times_last_year   |
| 123 environment_satisfaction   |
| 123 job_involvement            |
| 123 job_satisfaction           |
| 123 performance_rating         |
| 123 relationship_satisfaction  |
| 123 work_life_balance          |
| 123 num_companies_worked       |
| 123 total_working_years        |
| 123 years_at_company           |
| 123 years_in_current_role      |
| 123 years_since_last_promotion |
| 123 years_with_curr_manager    |
| AZ over_time                   |
| 123 standard_hours             |

Opción 2:

- Vaya al **Navegador de Base de Datos** (normalmente a la izquierda).
- Busque la base de datos **ibm\_hr**.
- Doble **Clic** en la tabla **employee\_master\_data**.
- Seleccione la carpeta **Diagramas** (Diagrams).

| employee_master_data           |
|--------------------------------|
| 123 employee_number            |
| 123 age                        |
| AZ gender                      |
| AZ marital_status              |
| AZ over18                      |
| AZ attrition                   |
| AZ business_travel             |
| 123 distance_from_home         |
| AZ department                  |
| AZ job_role                    |
| 123 job_level                  |
| 123 employee_count             |
| 123 monthly_income             |
| 123 monthly_rate               |
| 123 daily_rate                 |
| 123 hourly_rate                |
| 123 percent_salary_hike        |
| 123 stock_option_level         |
| 123 education                  |
| AZ education_field             |
| 123 training_times_last_year   |
| 123 environment_satisfaction   |
| 123 job_involvement            |
| 123 job_satisfaction           |
| 123 performance_rating         |
| 123 relationship_satisfaction  |
| 123 work_life_balance          |
| 123 num_companies_worked       |
| 123 total_working_years        |
| 123 years_at_company           |
| 123 years_in_current_role      |
| 123 years_since_last_promotion |
| 123 years_with_curr_manager    |
| AZ over_time                   |
| 123 standard_hours             |

- **Blueprint lógico:** piensa en el **ER** como el **plano arquitectónico** del edificio de datos. Cada **columna** es un **pilar** que soporta **análisis futuros**.



## Paso 8. Conexión Python-SQL

Integra **SQL** con **Python** para extracción inicial:

"""

TÍTULO: Extracción Modular de Datos desde PostgreSQL

OBJETIVO: Consultar la tabla maestra de empleados de forma segura y organizada.

DESCRIPCIÓN: El script utiliza variables de entorno para proteger las credenciales,

se conecta a la base de datos 'ibm\_hr', extrae una muestra de empleados y gestiona automáticamente el cierre de la conexión para optimizar recursos.

```
"""
```

```
import os
import psycopg2 # Conector para comunicarse con PostgreSQL
import pandas as pd # Herramienta para analizar y organizar datos en tablas
from dotenv import load_dotenv # Carga variables secretas desde un archivo .env
```

```
# Bloque de Configuración: Cargamos las credenciales ocultas del sistema
load_dotenv()
```

```
def obtener_conexion_db():
```

```
    """
```

```
    Establece el puente de comunicación con el servidor.
```

```
    Usa 'os.getenv' para leer los datos de acceso sin exponerlos en el código.
```

```
    """
```

```
    return psycopg2.connect(
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        host=os.getenv('DB_HOST'),
        port=os.getenv('DB_PORT'),
        database=os.getenv('DB_NAME')
    )
```

```
# Definimos cuántos registros queremos traer por defecto
limite = 10
```

```
def extraer_datos_empleados(limite):
```

```
    """
```

```
    Lógica principal para realizar la consulta y traer los datos a Python.
```

```
    """
```

```
    conn = None
```

```
    try:
```

```
        # Iniciamos la conexión llamando a la función anterior
```

```
        conn = obtener_conexion_db()
```

```
        # Definimos la instrucción SQL usando el límite solicitado
```

```
        query = f"""
```

```

SELECT *
FROM employee_master_data
LIMIT {limite}
"""

print("\n🚀 Iniciando ingesta PostgreSQL...")

# pandas ejecuta la consulta y transforma el resultado en una tabla
(DataFrame)
df = pd.read_sql(query, conn)

print("✅ ¡Datos cargados exitosamente!")
return df

except Exception as e:
    # Si algo falla (servidor caído, error de red), mostramos el motivo
    exacto
    print(f"❌ Error al conectar o procesar la base de datos: {e}")
    return None

finally:
    # Bloque de Cierre: Pase lo que pase, nos aseguramos de cerrar la
    conexión
    if conn:
        conn.close()

# Bloque de Ejecución: Punto de partida del programa
if __name__ == "__main__":
    # Llamamos a la función principal usando el límite definido arriba
    df_empleados = extraer_datos_empleados(limite)

    # Si la descarga fue exitosa, mostramos los resultados en pantalla
    if df_empleados is not None:
        print("\n Vista previa de los datos:")
        print(df_empleados.head())

```

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  HISTORIAL SQL  SUPERVISIÓN DE TAREAS  ...
🚀 Iniciando ingesta PostgreSQL...
✅ ¡Datos cargados exitosamente!

   age  gender marital_status over18 attrition  ... years_in_current_role  years_since_last_promotion  years_with_curr_manager  over_time  standard_hours
0   41  Female         Single      Y      Yes  ...                4                0                5          Yes            80
1   49   Male         Married      Y      No   ...                7                1                7          No            80
2   37   Male         Single      Y      Yes  ...                0                0                0          Yes            80
3   33  Female         Married      Y      No   ...                7                3                0          Yes            80
4   27   Male         Married      Y      No   ...                2                2                2          No            80

[5 rows x 35 columns]
PS C:\Users\User\AppData\Local\Programs\Microsoft VS Code>

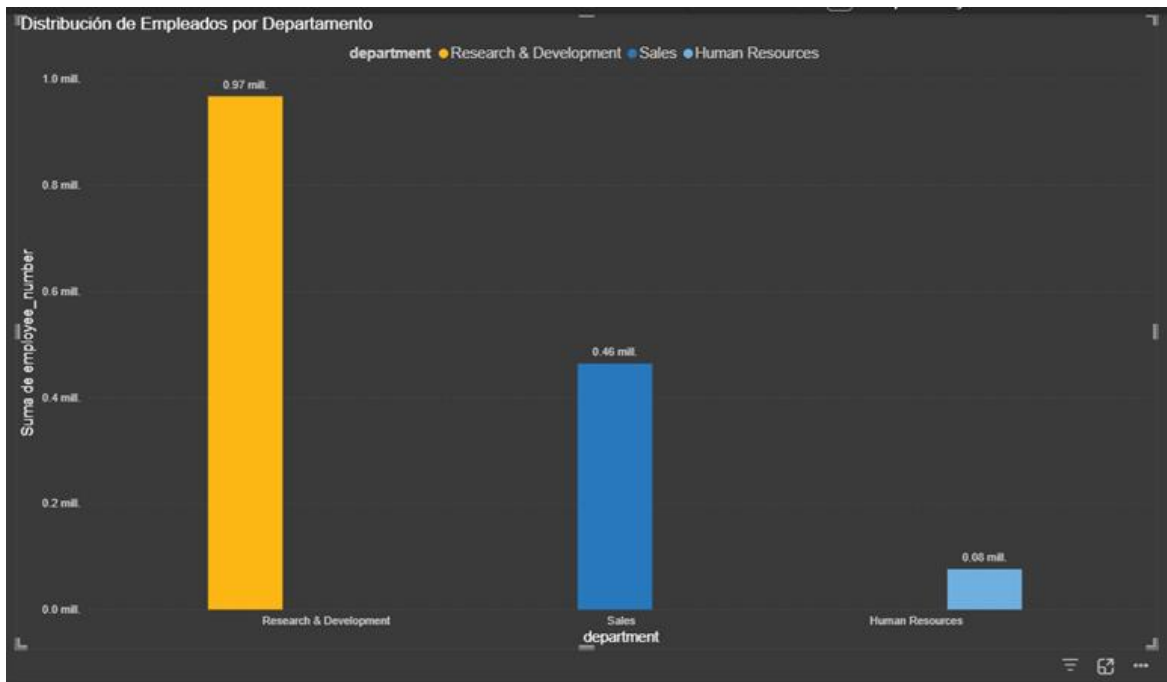
```

Lín. 22, col. 1 Espacios: 4 UTF-8 CRLF {} Python Python 3.14.3



## Paso 9. Conexión Power BI-SQL

- **Abrir Power BI Desktop:**
  - Localiza el ícono de **Power BI Desktop** en tu menú de inicio o barra de búsqueda.
  - Haz clic para abrir la aplicación.
  - Espera a que cargue la pantalla inicial con la opción de **Obtener datos**.
- **Seleccionar Fuente de Datos → PostgreSQL/SQL Server:**
  - En la pantalla principal, haz clic en **Obtener datos** (Get Data).
  - Se abrirá una ventana con múltiples opciones de fuentes.
  - En la opción **Base de Datos** selecciona **PostgreSQL**.
  - Haz clic en **Conectar**.
- **Configurar Servidor:**
  - En el cuadro de diálogo que aparece:
    - **Servidor:** escribe localhost (esto indica que la base está en tu propia máquina).
    - **Base de datos:** escribe **ibm\_hr**.
  - Haz clic en **Aceptar**.
- **Autenticación:**
  - **Usuario:** postgres
  - **Contraseña:** la que definiste al instalar **PostgreSQL** (ej. SaPassword2026!).
  - Haz clic en **Conectar**.
- **Importar la Tabla:**
  - **Power BI** mostrará un navegador con las tablas disponibles en la base **ibm\_hr**.
  - Selecciona la tabla que creaste (ej. employee\_master\_data).
  - Haz clic en **Cargar**.
  - **Power BI** importará los datos y los mostrará en el panel de campos.
- **Crear un Gráfico de Barras: Empleados por Department:**
  - En la vista de **Informe** (Report View), haz clic en el lienzo en blanco.
  - En el panel de **Visualizaciones**, selecciona el ícono de **Gráfico de barras agrupadas**.
  - Arrastra el campo department al eje **Categorías (Eje X)**.
  - Arrastra el campo employee\_number al eje **Valores (Eje Y)**.
    - **Power BI** automáticamente contará el número de empleados por departamento.
  - **Ajusta el formato:**
    - **Título:** "Distribución de Empleados por Departamento".
    - **Colores:** selecciona paleta corporativa o estándar.
    - **Etiquetas de datos:** activa para mostrar los valores sobre cada barra.



- **Insights:**
  - **Predominio del Core de Innovación:** Con 961 empleados, el área de **Research & Development (R&D)** es el **core de innovación** y **ventaja competitiva**. Operativamente, gestionar la **retención de talento** aquí es crítico, ya que cualquier cambio en este grupo de trabajo supone el mayor **impacto financiero** para la empresa.
  - **Motor Comercial Consolidado:** El **área comercial** (446 personas) sostiene el **crecimiento escalable** de **Research & Development (R&D)**, actuando como un **go-to-market** consolidado que garantiza ingresos estables.
  - **Estructura Administrativa Esbelta:** El departamento de **HH.RR.** destaca por su **estructura esbelta** (63 profesionales), gestionando una ratio aproximada de 1:23. Esto refleja **eficiencia administrativa**, aunque sugiere **implementar automatización** para evitar la **saturación del equipo**.
- **Validación del Paso 7:**
  - **SQL:** confirmas que la tabla integral está disponible en la base **ibm\_hr**.
  - **Power BI:** logras importar la tabla y visualizar un gráfico de barras.
  - **Resultado esperado:** un dashboard inicial que muestra la cantidad de empleados por departamento.